PARKVISION

Parking Lot Occupancy Detector

YOLOv8 · Computer Vision · Real-Time Detection

Yusuf Shahzad · ITAI 1378 – Computer Vision & AI · Tier 2

The Problem

30%

of urban traffic is caused
by drivers searching for parking

8 min

average time wasted per driver
circling a parking lot

$97B

lost annually in wasted fuel,
time & productivity

Core Issue: Parking lots have no real-time visibility into which spaces are occupied or available. Drivers must physically search every aisle, creating traffic congestion, fuel waste, frustration, and unnecessary emissions — all of which are entirely preventable with modern computer vision.

Solution Overview

ParkVision uses YOLOv8 to detect occupied & empty spots from overhead camera images in real-time.

INPUT

Overhead parking lot image (640×640)

DETECT

YOLOv8s runs object detection

CLASSIFY

Each space labeled occupied / empty

OUTPUT

Visual overlay + availability count

✓ **Achieved:** 99.5% mAP@0.5 · 99.8% Precision · 99.8% Recall · ~17ms inference

Technical Approach

Model	YOLOv8s (Ultralytics)	Small model — fast, accurate, pre-trained on COCO
Framework	PyTorch 2.0 + Ultralytics	Industry-standard CV stack
CV Task	Object Detection	Bounding box regression + classification
Input	640 × 640 px images	Overhead parking lot camera frames
Output	Bounding boxes + class labels	occupied / empty with confidence score
Training	Google Colab T4 GPU	50 epochs, early stopping, batch 16

Dataset

Source

PKLot Dataset

Roboflow Universe · sagitova-aliya/pklot-qesrf

~12K

Labeled Images

2

Classes

Dataset Split

Train 70%

Valid 15%

Test 15%

Classes

empty · Available parking space

occupied · Space with a vehicle

System Diagram

CAMERA	PREPROCESS	YOLOV8s	CLASSIFY	OVERLAY
Overhead image 640×640 px	Resize + normalize RGB frame	Detection model 73 layers	occupied / empty per box	Red / green boxes + count HUD

Actual Performance

99.5%	99.8%	99.8%	17ms	57×
mAP@0.5	Precision	Recall	Inference	Faster than target

Results

99.5%

mAP@0.5
(Target ≥ 85%)

99.8%

Precision
(Near perfect)

99.8%

Recall
(Near perfect)

17ms

Inference Speed
(Target < 1000ms)

Week-by-Week Plan

Week 10	Dataset setup, Colab env, repo structure	Dataset ready	✓
Week 11	Train YOLOv8 baseline model	Model running	✓
Week 12	Fine-tuning, augmentation, validation	99.5% mAP achieved	✓
Week 13	Visual overlay demo (red/green), backup video	Demo notebook ready	
Week 14	Final docs, README update, slide polish	Fully documented	
Week 15	In-class presentation & live demo	Present	

Challenges & Backup Plans

Medium	**Occluded / overlapping vehicles** → Data augmentation, try YOLOv8m	Mitigated
Medium	**Dataset mismatch (different lots)** → Supplement with additional Roboflow sets	Mitigated
Low	**Google Colab GPU quota limits** → Switch to Kaggle (30 hrs/week free GPU)	
Medium	**Overfitting on training set** → Early stopping + dropout regularization	Mitigated
Low	**Live demo failure during class** → Pre-recorded backup video of full demo	

Resources & Tools

Compute	Google Colab (T4 GPU)	Free GPU — ~17ms inference	$0
Model	Ultralytics YOLOv8s	Open-source, Apache 2.0	$0
Dataset	PKLot via Roboflow Universe	~12,000 labeled images	$0
Framework	PyTorch 2.0	CUDA acceleration	$0
IDE	Jupyter / Google Colab	Notebooks committed to GitHub	$0
Repo	GitHub	YusufShahz/ITAI1378_Midterm_ParkVision	$0

Total Project Cost: $0.00 — 100% open-source stack